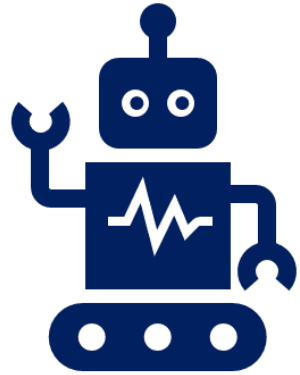




Agentic AI in Recommender Systems

Agent

An agent is just something that acts

(agent comes from the Latin agree, to do)

It is the capability beyond doing normal tasks such as sense the operating environment, take decision etc.

Rational Agent

A rational agent is one that acts so as to achieve the **best outcome** or when there is **uncertainty**, the **best expected outcome**.

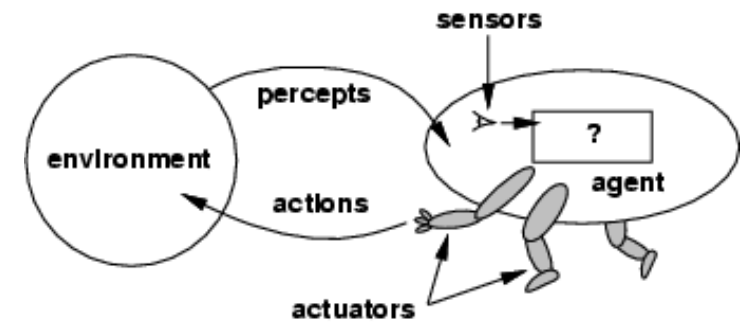
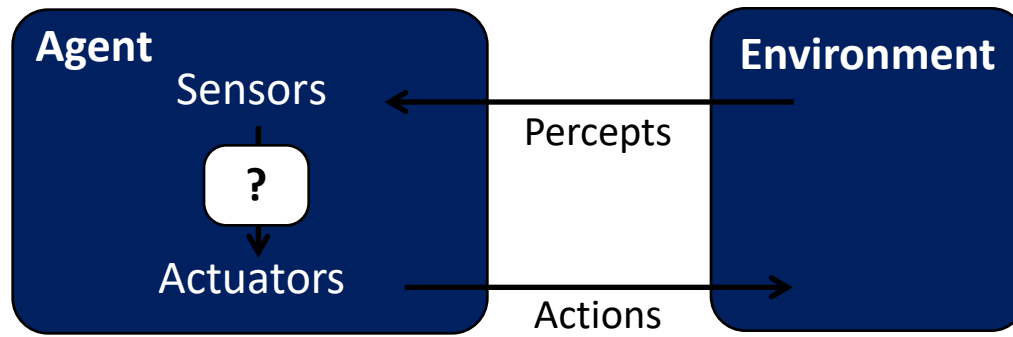
Or

It is one that does the right thing.

Agents and Environments

An **agent** perceives the **environment** through **sensors** and acting upon that environment through **actuators**.

Environment could be everything



Agents interact with environments through sensors and actuators

Agents and Environments (contd..)

Examples:

Human Agent

Robotic Agent

Software Agent

•

•

•

Many more

Agents and Environments (contd..)

Percept – The content an agent's sensors are perceiving.

An agent **percept sequence** is the **complete history** of everything that agent has ever perceived.

An agent choice of actions at any given instant can depend on its **built-in knowledge** and on the entire **percept sequence** observed to date.

Agents and Environments (contd..)

Mathematically, **agent behaviour is described by the Agent Function:** that maps any given precept sequence to an action.

Agent Function *can be tabulated in a table form for any given agent placed in an experimental environment, constructing the table by trying all possible percept sequences and recording which actions the agent does in response.*

Agents and Environments (contd..)

Agent function is implemented by an agent program.

Agent Function is an abstract mathematical description.

Agent Program is a concrete implementation, running within some physical system

The Structure of Agents

Agent = Architecture + Program



Agentia World

Imagine a world where everything is an AI agent, from your coffee machine to your car, from businesses to entire cities.

Picture a world transformed into **Agentia**—a dynamic, living network of intelligent AI agents seamlessly integrated into our daily lives.

Agentia is more than digital—it's also physical, brought to life by robots that serve as embodied agents interacting with and enhancing our physical world.

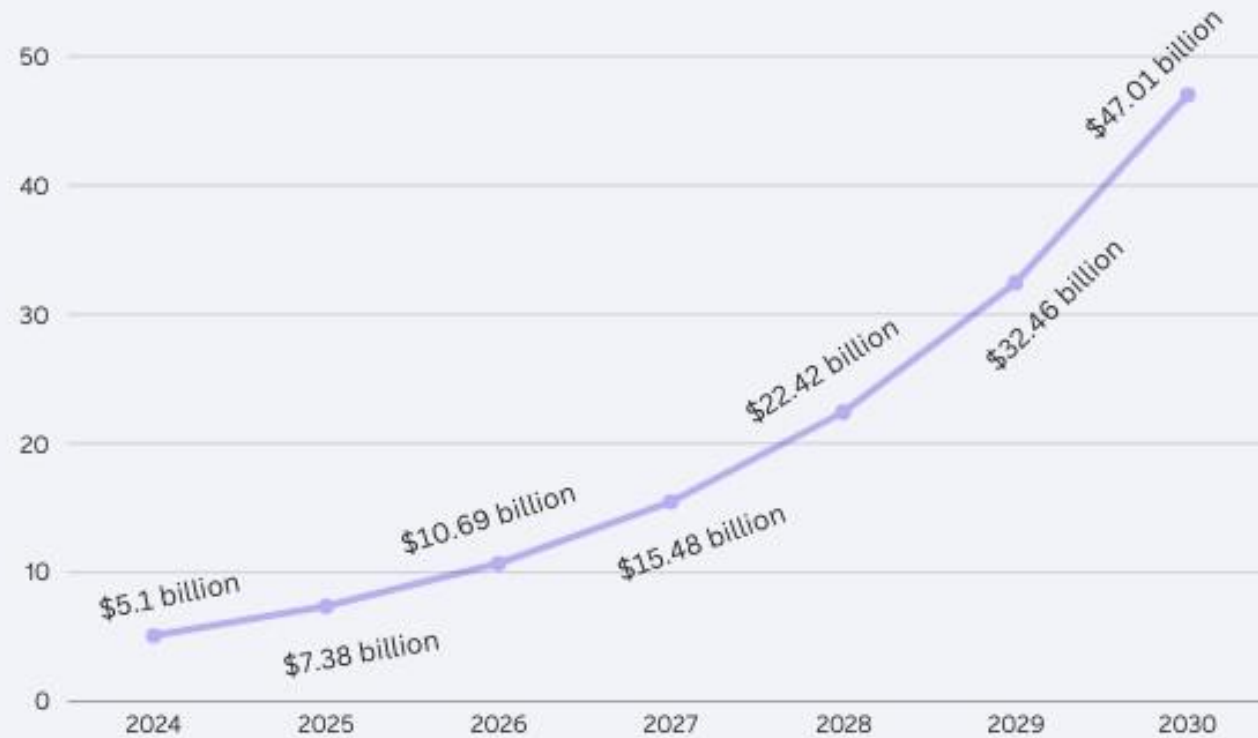
Agentia World



AI Agents Market Size



The AI agents market is estimated to be valued at **\$7.38 billion** and will continue to grow at a compound annual growth rate of **44.8%** and reach **\$47.1 billion by 2030**.



What Are Agentic AI Systems?

Agentic AI refers to **artificial intelligence systems** designed to act autonomously, capable of performing tasks, making decisions, and interacting with their environments without requiring direct human intervention.

What is an AI Agent?

An autonomous entity that perceives, decides, and acts in digital or physical environments.

Types:

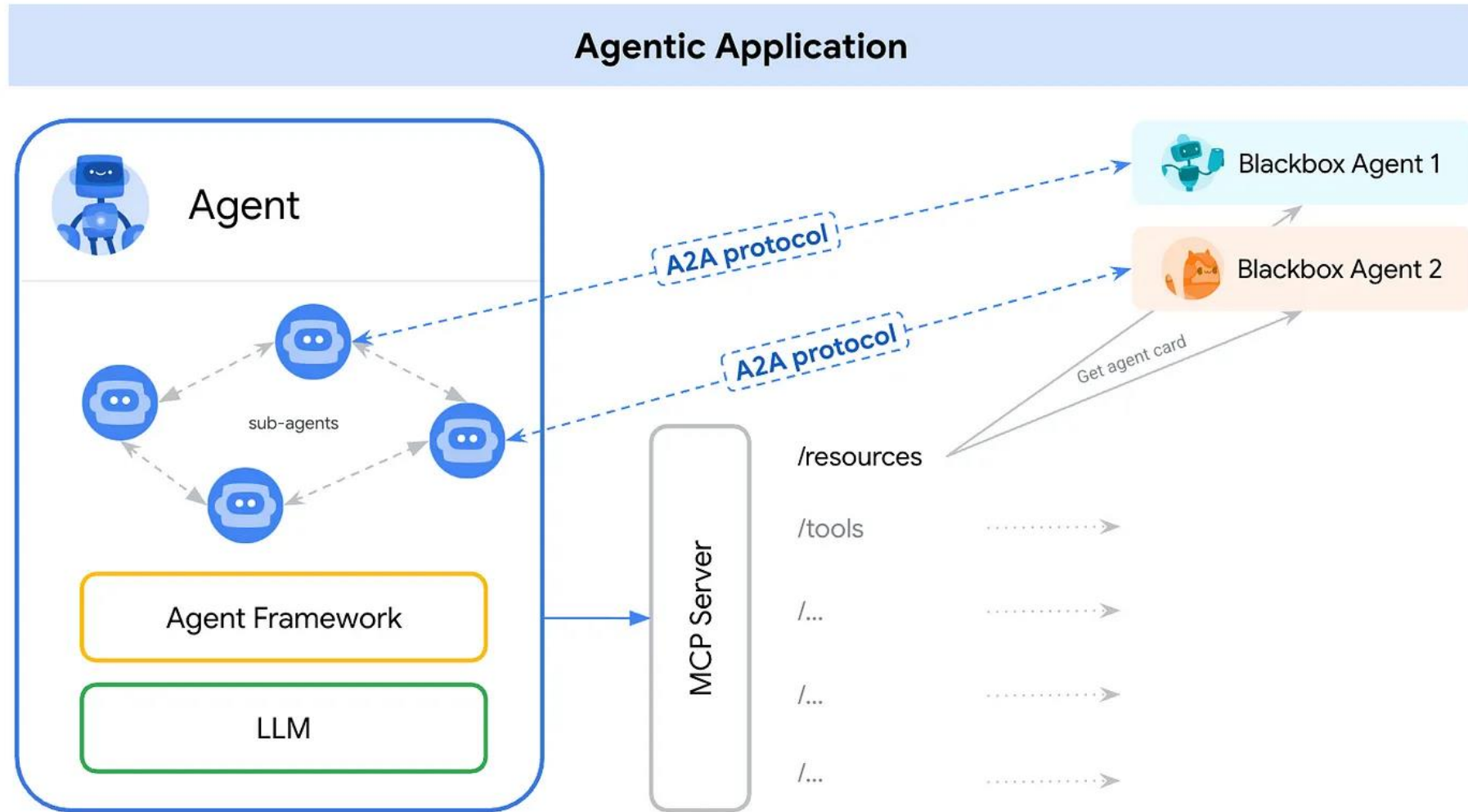
Software Agents: Operate in digital systems (e.g., chatbots, workflows).

Physical Agents: Interact with the physical world (e.g., robots, vehicles).

Agentic AI systems differ from traditional AI in several keyways:

- Autonomy:** They can function without constant human oversight, making them ideal for scenarios where human intervention is impractical or unnecessary.
- Flexibility:** These systems can adapt to new data and circumstances, handling unexpected inputs and changes in their environment without manual oversight.
- Problem-solving:** With advanced reasoning, planning, and goal-setting abilities, agentic AI can tackle complex, multi-step problems, often beyond the capabilities of traditional AI.
- Creativity:** Agentic AI can explore novel solutions and hypotheses, potentially leading to breakthroughs in various fields, including drug discovery and precision medicine.

The Technology Architecture for Agentia World



The Technology Architecture for Agentia World

This diagram represents the technology architecture for an "Agentic Application" in the envisioned "Agentia World":

- 1. Agentic Application:** The environment where agents operate, facilitating their interactions and task execution.
- 2. Agent:** The central AI or system coordinating activities, interacting with sub-agents and external blackbox agents.
- 3. Sub-Agents:** Specialized agents within the Agentic Application, communicating with the main Agent and each other to handle specific tasks, indicating a modular structure.
- 4. Agent Framework:** A foundational layer (in yellow) providing the structure and protocols for agent operations, including libraries or rules for cohesive functionality.
- 5. LLM (Large Language Model):** Highlighted in green, the LLM powers the agents' intelligence for natural language processing, decision-making, and task execution. It's integrated within the Agent Framework.

6.MCP (Model Context Protocol): MCP is a protocol that manages the context and state of the LLM during interactions. It facilitates the exchange of contextual data (e.g., /resources, /tools) between the Agent Framework and external systems, ensuring the LLM maintains relevant context for tasks.

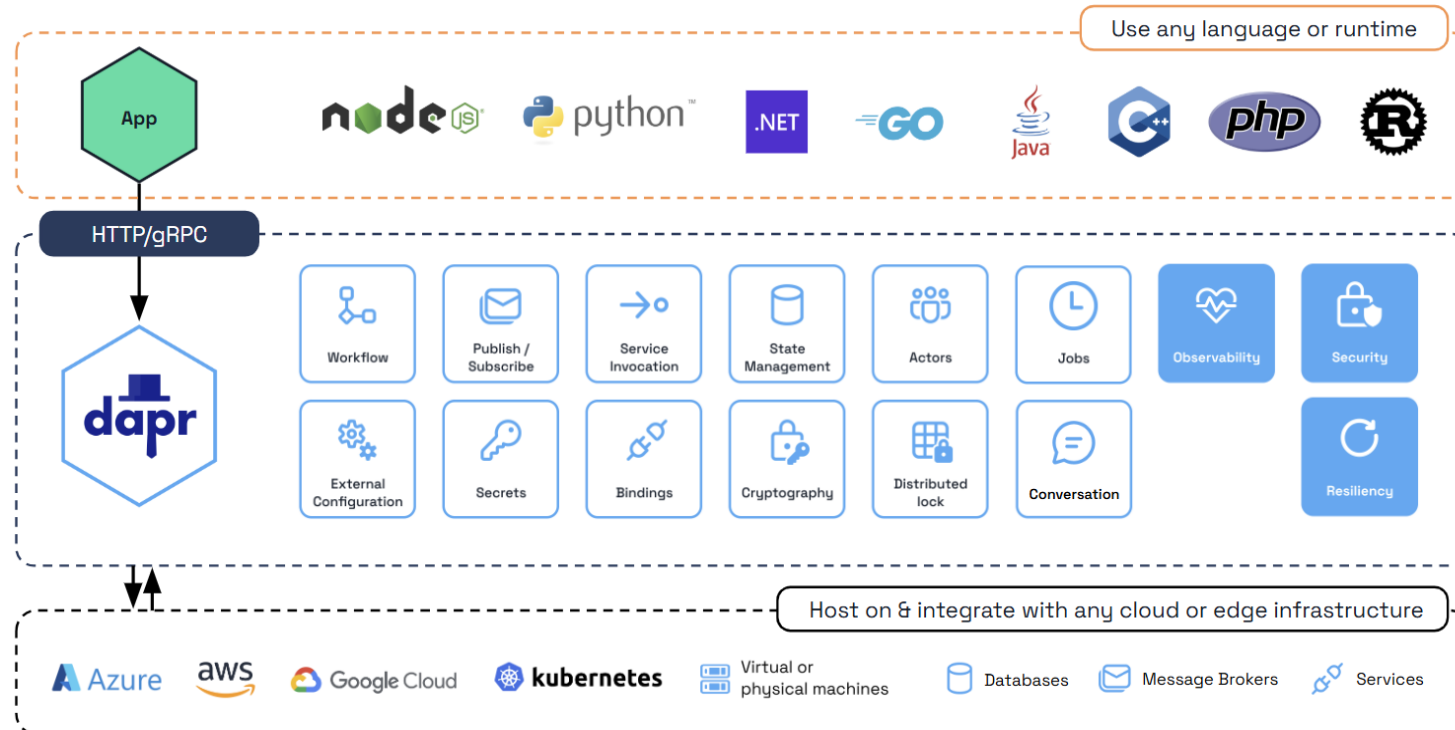
7.Resources/Tools: External inputs (e.g., /resources, /tools) accessed via the MCP. These could include databases, APIs, or services that agents use, with the MCP ensuring the LLM understands the context of these resources.

8.Blackbox Agents (1 and 2): External agents interacting with the main Agent using the A2A (Agent-to-Agent) protocol. They likely represent third-party systems collaborating with the main Agent, with interactions like "GET agent card" indicating specific data or capability requests.

9.A2A Protocol: The Agent-to-Agent protocol standardizing communication between the main Agent and external blackbox agents for secure and efficient collaboration.

10. Distributed Application Runtime (Dapr):

- **Purpose:** Simplifies distributed systems with standardized building blocks (state, pub/sub, workflows).
- **Implementation:** Runs as a sidecar container, managing state (Redis, CockroachDB), messaging (RabbitMQ, Kafka), and workflows.
- **Optional:** Dapr Agents and Dapr Workflows for advanced orchestration.
- **Why It Matters:** Ensures resilience. abstracts infra complexity. and future-proofs the system.

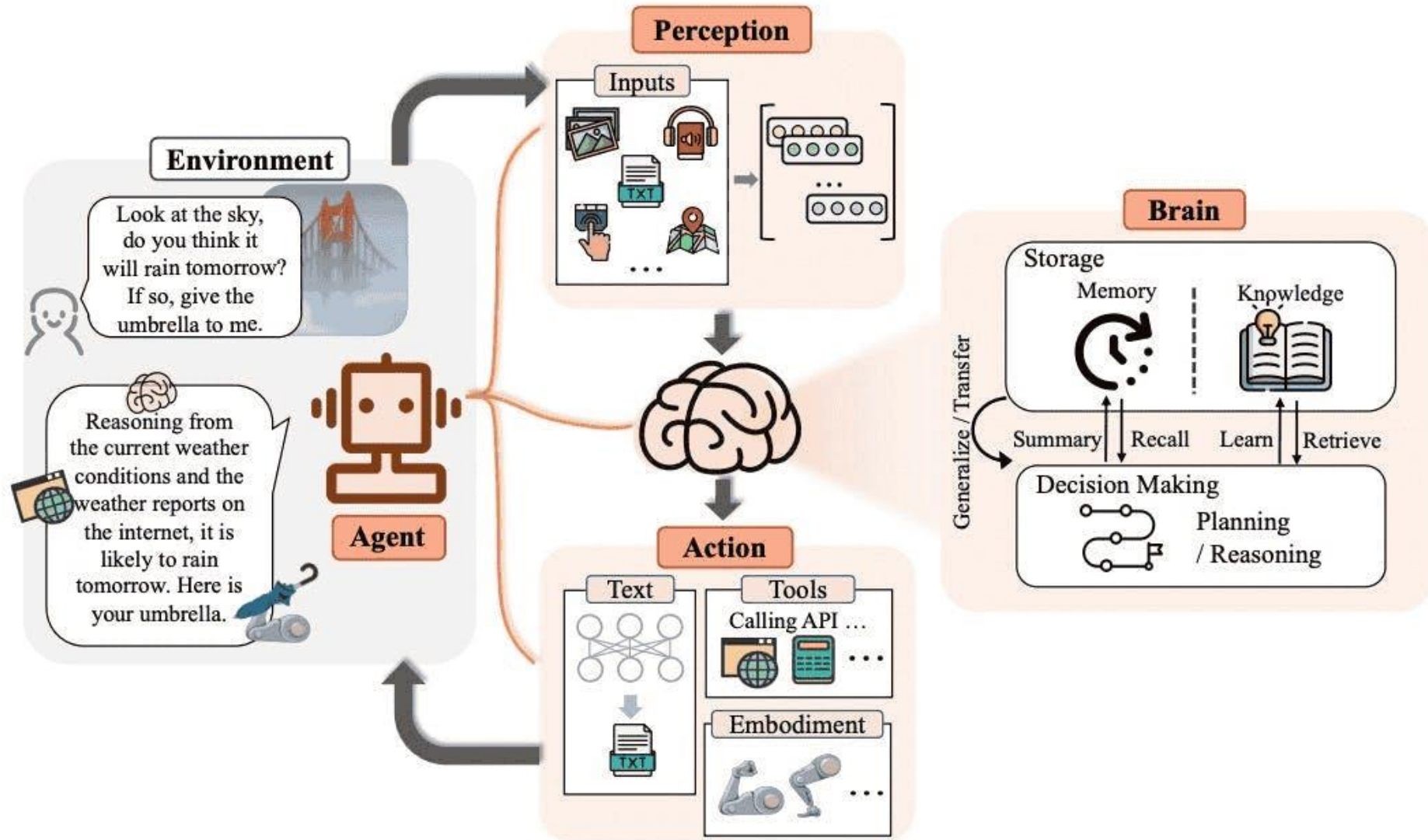


Workflow Summary:

- The **Agent** delegates tasks to **Sub-Agents** within the **Agentic Application**.
- The **Agent Framework**, powered by the **LLM**, provides the operational structure. For example, OpenAI's Agents SDK, Dapr Agents, LangGraph, AutoGen, ADK, etc.
- The **MCP (Model Context Protocol)** ensures the LLM maintains proper context while accessing external **Resources/Tools**.
- The **Agent** collaborates with **Blackbox Agents** via the **A2A Protocol** for external interactions.

This architecture supports a modular, context-aware system where agents, powered by an LLM, operate cohesively using the MCP and A2A Protocols to manage interactions with external resources and agents in the "Agentia World."

Core Components of Agentic AI Architectures



Perception Module

The Perception module is responsible for collecting and interpreting data from the environment. This involves utilizing sensor technologies and data ingestion pipelines to capture diverse inputs.

For instance, in autonomous driving, systems employ cameras, LiDAR, and radar to gather visual and spatial information, which is then processed to understand the vehicle's surroundings. Recent advancements propose integrating perception and decision-making processes using Transformer architectures to handle multimodal data efficiently.

Decision-Making Engine

The Decision-Making Engine employs algorithms and models to guide actions and assess potential risks. Reinforcement learning is a prominent approach where agents learn optimal behaviors through trial and error, receiving feedback from their actions.

Heuristic-based decision trees offer structured pathways for decision-making by applying predefined rules. Emerging research emphasizes the importance of robust reasoning and planning capabilities in AI agents to achieve complex goals.

Learning Mechanism

The Learning Mechanism enables agents to adapt over time through various machine-learning techniques:

- Supervised Learning: Training on labeled data to recognize patterns and make accurate predictions.
- Unsupervised Learning: Identifying hidden structures in unlabeled data, facilitating the discovery of underlying patterns.
- Reinforcement Learning: Learning optimal actions by interacting with the environment and receiving feedback through rewards or penalties.

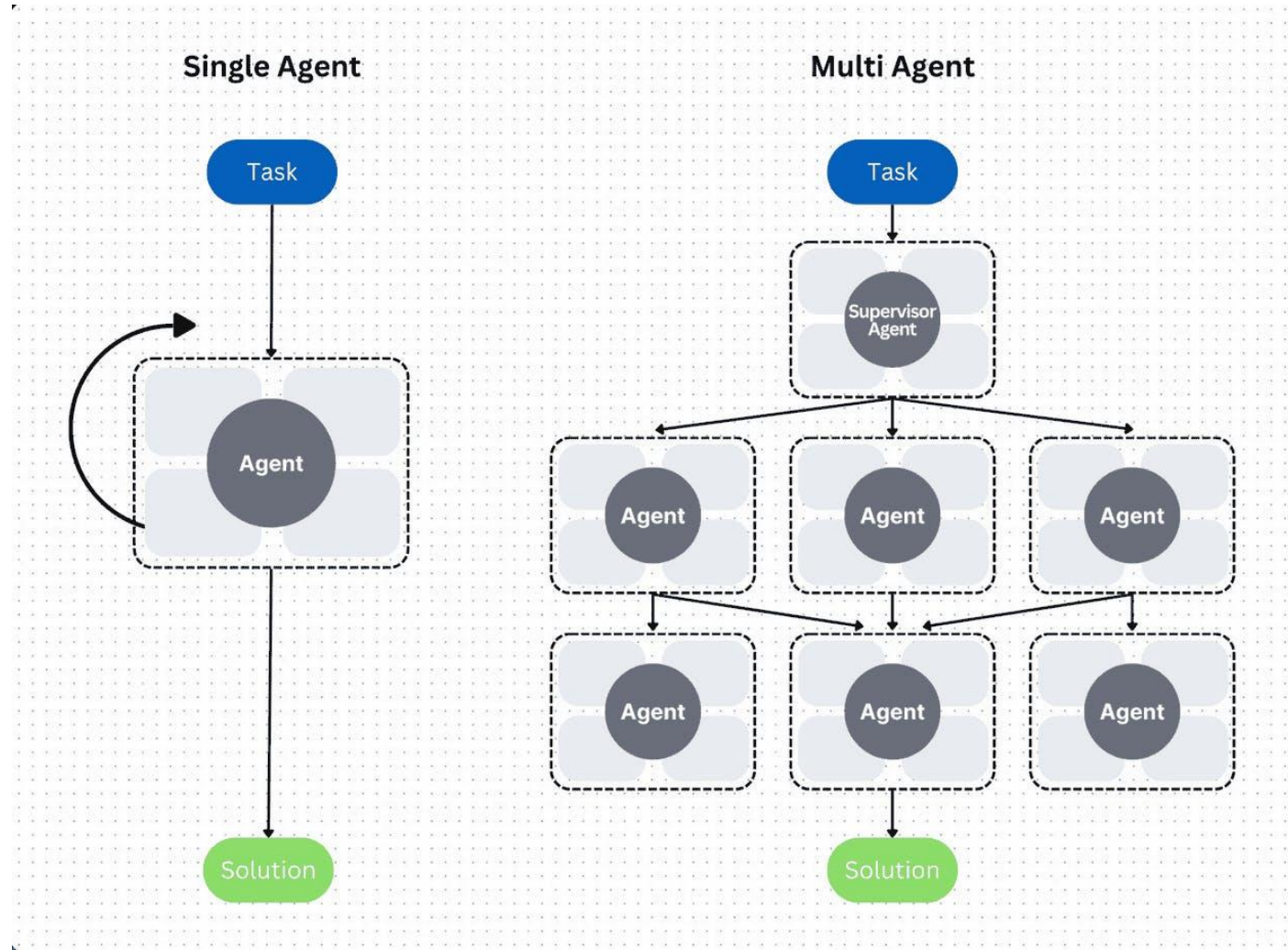
Integrating these learning paradigms allows agents to refine their behaviors and improve performance across diverse tasks.

Action Module

The Action Module executes decisions and interfaces with real-world or simulated environments. In robotics, this involves controlling actuators to perform physical tasks.

In digital systems, it may entail initiating processes or communicating with other software components. The effectiveness of this module depends on the precision and timeliness of actions, ensuring that decisions lead to desired outcomes.

Single-Agent vs. Multi-Agent Systems



Popular Frameworks for Building AI Agents

Table 1: Comparison of different Agentic AI frameworks

Framework Name	Open Source	GitHub Stars	Key Features	Ideal Use Case
AutoGen	Yes	35k+	It supports multi-agent collaboration and integrates LLMs, asynchronous messaging, and code execution capabilities.	Building complex AI applications with multi-agent collaboration.
CrewAI	Yes	22k+	Multi-agent collaboration, role-based design, integration with external tools, customizable workflows.	Automating complex workflows across various industries.
AgentGPT	Yes	32k+	Customizable agent creation, autonomous goal pursuit, plugin support, and user-friendly setup.	Automating tasks such as content generation, customer support, and data analysis.
MetaGPT	Yes	45k+	A multi-agent framework assigns diverse roles to agents, utilizes LLMs and SOPs for collaboration, and enhances efficiency in complex tasks.	Software development and engineering, project management, and coordination

Agentic AI in RS System

Four-Level Evolution of Recommender Systems

Level	Name	Description	Key Characteristics
0	Traditional Recommender Systems	Systems rely on static algorithms and historical data to suggest items.	• Rule-Based Processing: Uses fixed rules, collaborative filtering, or content-based methods. • Limited Contextual Understanding: Operates solely on past user behavior without real-time adjustments. • One-Way Interaction: Provides recommendations in a non-interactive, one-off manner.
1	Advanced Recommender Systems	Deep learning advances enhance personalization with historical and real-time data.	• Data-Driven Adaptation: Uses learning models to update recommendations based on new information. • Feedback Integration: Incorporates user feedback to refine suggestions over time. • Enhanced Personalization: Provides more accurate and context-aware recommendations while following predefined model structures.
2	Intelligent Recommender Systems	These systems actively engage users to refine their understanding of preferences.	• Interactive Engagement: Initiates clarifying dialogues and solicits additional input. • Multi-Modal Input Processing: Integrates inputs beyond text (e.g., images, behavioral signals). • Dynamic Adaptation: Adjusts recommendations in real-time based on user context.
3	Agentic Recommender Systems	Fully autonomous agents that not only provide recommendations but also self-improve and evolve.	• Autonomous Decision-Making: Uses planning and optimization to proactively shape recommendation strategies. • Continuous Self-Evolution: Updates models and behaviors based on internal and external feedback. • Comprehensive Memory & Multi-Modal Perception: Integrates long-term user data, contextual cues, and multiple input types. • Proactive and Reactive Interactions: Balances immediate responses with strategic actions.

Background

With the rapid growth of online services, recommender systems (RS) have become essential for addressing users' information needs and alleviating information overload.

These systems provide personalized recommendations across various domains, including e-commerce, movies, music, etc.

Despite the diversity of recommendation tasks such as top-K recommendation and sequential recommendation, the core objective remains consistent: to predict a user's preferences for each candidate item and generate a ranked list tailored to the user.

However, current RSs still face several significant limitations in meeting diverse user needs:

First, current RSs typically rely on ID-based features that work only within specific domains or platforms. Their inability to integrate open-domain knowledge, such as common sense reasoning and cross-platform behavioral patterns, significantly constrains their capacity to interpret and model user interests in a broader context

Background

Second, current methods typically optimize well-defined engagement metrics derived from historical interaction data (e.g., click-through rates and purchase histories). Although such methods can be effective for localized objective functions, they often conflate observable behaviors with latent user intent, since implicit feedback mechanisms cannot distinguish transient actions from enduring preferences

Consequently, these models exhibit two major limitations: **(i) lack of transparency regarding preference attribution, which impairs interpretability, and (ii) oversimplification of the multifaceted motivations that guide user behavior, especially in scenarios requiring temporal or situational adaptation.**

As a result, these implicit modeling frameworks fail to capture the causal relationships between dynamic user states and subsequent decision-making processes

Background

Finally, most traditional RSs operate in a largely static, one-directional manner, providing users with minimal opportunities to iteratively refine suggestions through natural language or real-time feedback.

This unidirectional flow diverges from established human-computer interaction principles, which emphasize interactive, adaptive dialogue to uncover user preferences.

Although conversational RSs have begun to address this issue, they remain limited in their ability to integrate open ended natural language understanding with personalized ranking, particularly in scenarios that require multiple rounds of clarification to resolve ambiguous user queries

Agentic AI In RS

Recent advances in Large Language Models (LLMs) and Multimodal LLMs (MLLMs) have greatly improved language comprehension and cognitive processing [24, 39].

With stronger reasoning and planning abilities, (M)LLM-based agents can interpret human language, devise strategies, and execute complex tasks.

These breakthroughs offer new avenues for enhancing RSs' adaptability, personalization, and user-centricity. The rapid surge in LLM-driven RS research is evident from the 290 references cited in a recent survey on this topic [17, 31, 32], along with numerous influential papers in the field (e.g., [44]).

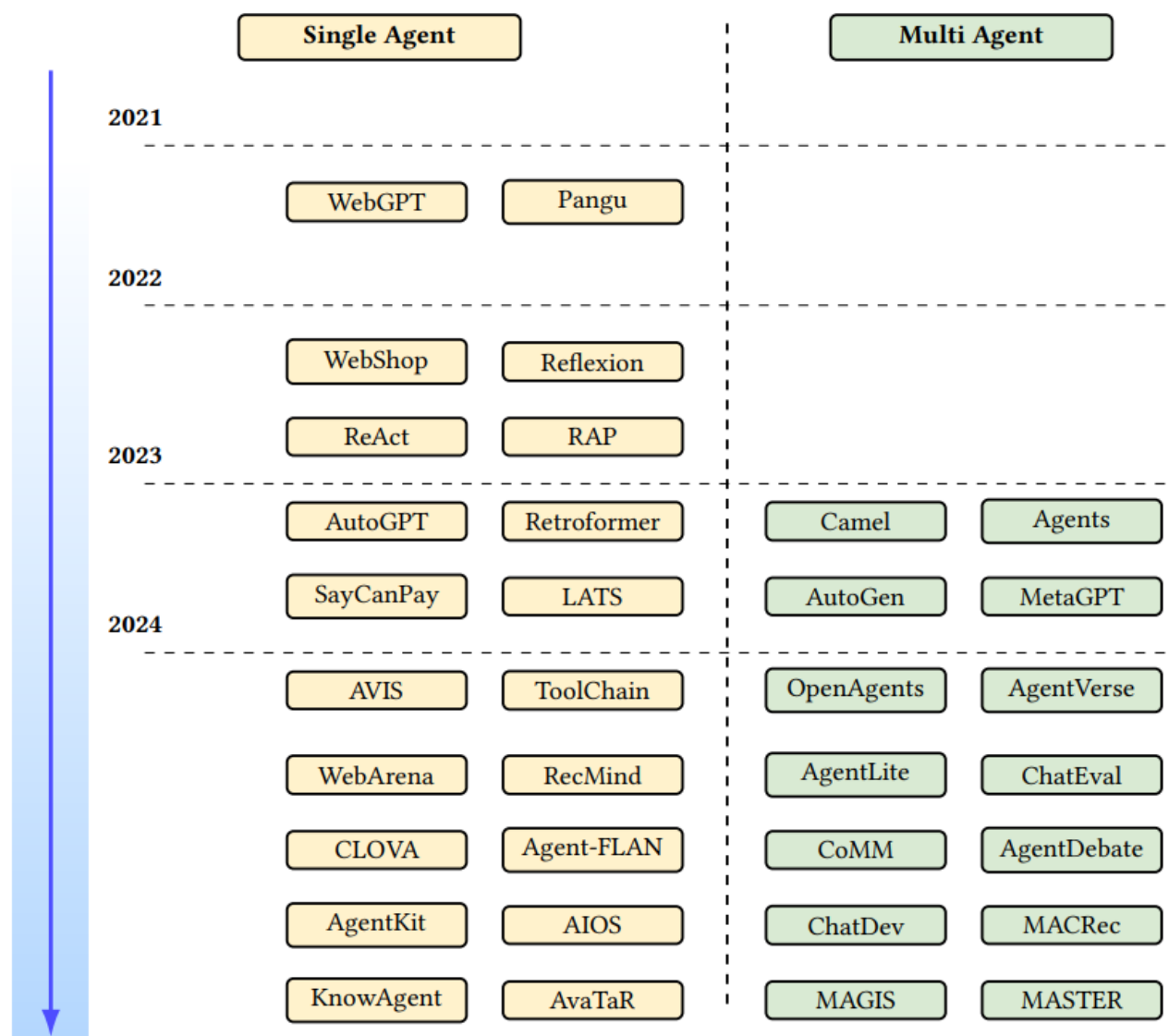


Figure 1: The rising trend in the research field of LLM-based Agents. We categorize current work into single-agent and multi-agent categories.

LLM-based Single-Agent Systems

Single-agent systems leverage a unified model that integrates multiple interdependent modules.

The memory component acts as a structured repository that stores and retrieves contextually relevant information, such as user preferences and historical interactions [93].

This persistent memory is crucial for maintaining coherent, long-term interactions and forms the foundation for personalization in recommendation settings.

Recent research has identified approaches such as task decomposition, multi-plan selection, external module-aided planning, reflection and refinement, and memory-augmented planning [21].

These techniques enable an agent to break down complex tasks, select and refine strategies based on evolving contexts, and leverage external knowledge sources.

Frameworks like ReAct [80] and Reflexion [52] exemplify how interleaving reasoning with concrete actions—such as web-browsing or tool invocation—can significantly improve system robustness and adaptability.

LLM-based Single-Agent Systems

Beyond internal cognitive processes, these agents increasingly rely on tool utilization to interface with external data and services.

Systems like WebGPT [37] illustrate the effectiveness of using external modules (e.g., web search engines) to retrieve real-time information.

Other works, such as Retroformer [81] and AvaTaR [75], further optimize these interactions through policy gradient optimization and contrastive reasoning, respectively, to fine-tune tool usage and enhance performance over time.

LLM-based Multi-Agent Systems

In contrast, LLM-based **multi-agent systems** emphasize collaboration among diverse autonomous agents.

These systems are designed to mimic complex human workflows by facilitating inter-agent communication, task specialization, and coordinated decision-making.

Frameworks such as CAMEL [28] and AutoGen [74] demonstrate how agents with distinct roles can interact to solve problems more efficiently than a single, monolithic agent.

Further advancements are seen in approaches like MetaGPT [15] and AgentLite [34], which incorporate meta-programming techniques and lightweight libraries to dynamically allocate roles and coordinate complex workflows.

Why Agentic Recommender Systems Now?

Recent advances in RSs have largely focused on enhancing interaction capabilities, with most research efforts still operating at the Advanced RSs (Level 1) and Intelligent RSs (Level 2) stages as shown in Table 1.

However, they remain fundamentally reactive, relying on predefined model architectures and user-driven feedback loops.

The next frontier, Agentic RSs (Level 3), aims to move beyond reactive engagement to autonomous, adaptive, and proactive recommendation strategies, which is increasingly feasible due to recent breakthroughs in (M)LLMs.

Leveraging (M)LLMs for Recommendation: The integration of LLMs introduces agent-like capabilities such as planning, memory retention, and in-context learning, enabling adaptive and evolving recommendation strategies. Unlike traditional systems that require explicit re-training, LLM-based agents can dynamically refine recommendations based on sequential user interactions and external contextual cues. Additionally, collaborative multi-agent systems can further enhance recommendations by enabling multiple AI agents to exchange information, reason collectively, and optimize decision-making.

Expanding Information Modalities: RSs primarily rely on ID-based and textual information, limiting their ability to fully understand user preferences. In contrast, multi-modal agentic systems can process diverse input signals, including images, audio, structured metadata, and behavioural cues, leading to richer and more context-aware recommendations. Thus, agentic systems can capture holistic user intent, bridging the gap between implicit and explicit preference signals.

Evolving User Interfaces: From Passive to Proactive Recommendation: Traditional recommendation paradigms primarily function as passive systems, responding to user queries with static suggestions. Conversational recommenders improve engagement but still rely on user-initiated interactions. Agentic systems introduce a proactive user experience, where AI-powered multi-modal agents continuously adapt, predict user needs, and autonomously refine recommendations before explicit queries occur. This shift not only enhances user satisfaction but also opens the door for highly personalized, real-time, and contextually aware recommender systems.

Formulation

An Agentic Recommender System [86, 90] is a system in which agents autonomously generate personalized recommendations by interacting with users and adapting to their preferences over time.

Formally, it can be defined as a **tuple** (U, I, A, E, R) , where U is the set of users, I is the set of items, A is the set of agents, E is the set of environmental contexts and $R : U \times E \times A \rightarrow \mathcal{P}(I)$ is the recommendation function that maps users, contexts, and agents to a probability distribution over items $\mathcal{P}(I)$.

Each agent $a \in A$ operates autonomously by perceiving the state $s = f(u, e)$, making decisions based on its policy $\pi_a(s)$, and learning from user feedback to optimize an objective function, maximizing expected user utility:

$$\max \pi_a \mathbb{E} [U(u, R(u, e, a)) \mid \pi_a] .$$

The key characteristics of such a system include autonomy, adaptability, and enabling agents to provide dynamic and personalized recommendations through continuous learning and user engagement.

Symbol	Description
U, I, A, E	Users, items, agents, environments
$R : U \times E \times A \rightarrow P(I)$	Recommendation function
$s = f(u, e)$	User state representation
$\pi_a(s)$	Agent policy
$P(I)$	Item distribution
$H(u, t)$	User interaction history
$C(u, t)$	Contextual factors
$X(u, t)$	External signals
$P(u, t)$	User profile
$M(u, t)$	Memory function
$\mathcal{A}(s, a)$	Action selection function

Table 2: Summary of notations used in agent-based RSs.

The User Profiling module:

The User Profiling Module is dedicated to constructing comprehensive profiles, such as behaviours for each user. The function can be define as $P : U \times T \rightarrow S$, where $P(u, t)$ represents the evolving profile of user u at time t .

This profile is dynamically updated based on historical interactions $H(u, t)$, contextual features $C(u, t)$, and external signals $X(u, t)$, modeled as:

$$P(u, t) = f(H(u, t), C(u, t), X(u, t); \theta_p)$$

To adapt to new user behaviours, profile updates incrementally as:

$$P(u, t + 1) = P(u, t) + \eta \cdot \Delta P(u, t)$$

where $\Delta P(u, t)$ represents changes based on recent interactions, and η controls the update rate.

The user profiling module employs machine learning techniques to adaptively refine user profiles over time.

The Planning module:

The Planning Module empowers agents to formulate strategic decisions regarding which items to recommend. Using the user profiles from the User Profiling Module and considering the current environmental context $e \in E$, the module is defined as:

$$s = f(u, e),$$

where $f : U \times E \rightarrow S$ maps users and contexts to a state space S . for each user-agent pair.

This module functions as the core of the decision-making of the Agentic Recommender System, the Planning Module leverages advanced optimization techniques, such as Markov Decision Processes (MDPs) and reinforcement learning, to ensure that decisions are both rational and aligned with user objectives.

The Memory module:

The Memory Module functions as a dynamic storage system that retains historical data on user interactions and feedback.

It serves as a critical component for enabling the Agentic Recommender System to build continuity and context awareness over time. Formally, it maintains a memory function $M : U \times T \rightarrow M$, where:

$$M(u, t) = g(H(u, t), C(u, t); \theta_M)$$

By storing and retrieving historical data, this module ensuring that future recommendations are informed by accumulated insights.

The Memory Module is designed to support both short-term and long-term memory functionalities.

Short-term memory stores recent interactions, enabling the system to adapt to immediate user needs and preferences.

In contrast, long-term memory archives broader behavioural patterns, which are crucial for understanding shifts in user behaviour over time.

The Action module:

The Action Module is responsible for executing the decisions made by the Planning Module, dynamically selecting and delivering recommendations to users.

Given a user $u \in U$, an agent $a \in A$, and an environmental state $e \in E$, the system defines an action selection function $A : S \times A \rightarrow P(I)$, where: $A(s, a) = \pi_a(s)$

where $\pi_a(s)$ represents the agent's policy for selecting a probability distribution over items $P(I)$, given the current state $s = f(u, e)$.

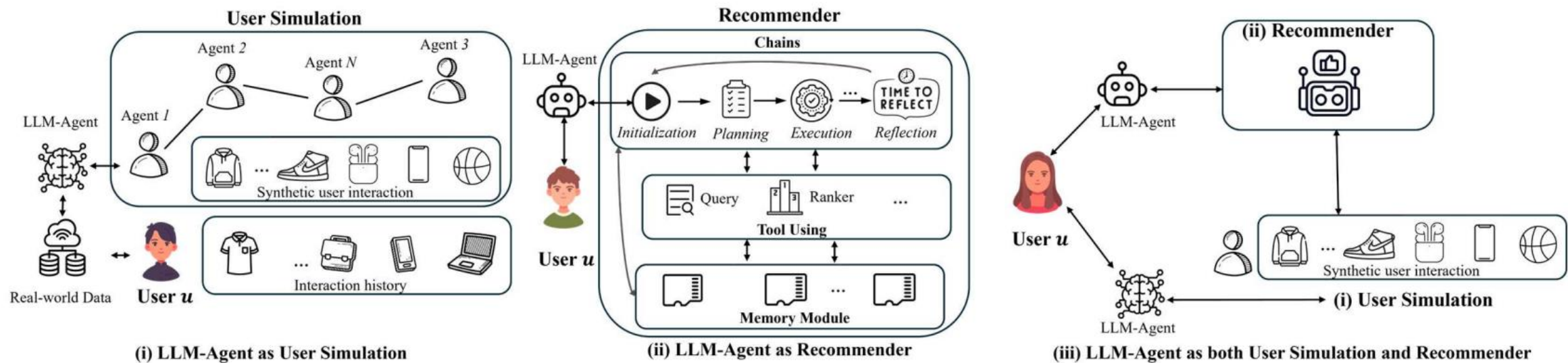


Figure 2: Different types of personalized LLM-based agents in LLM-ARS, where (i) LLM-Agent simulates user behavior, (ii) LLM-Agent acts as a recommender, and (iii) LLM-Agent functions as both user simulation and recommender.

Multi-agent

```
# Install dependencies
!pip install pandas scikit-learn

# Imports
import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.metrics.pairwise import cosine_similarity

# Sample data
movies = pd.DataFrame({
    'movie_id': [1, 2, 3, 4, 5],
    'title': ['The Matrix', 'John Wick', 'Inception', 'Interstellar', 'The Dark Knight'],
    'genre': ['Action Sci-Fi', 'Action Thriller', 'Action Sci-Fi', 'Sci-Fi Drama', 'Action Drama']
})

ratings = pd.DataFrame({
    'user_id': [1, 1, 2, 2, 3, 3, 4],
    'movie_id': [1, 3, 2, 4, 1, 5, 3],
    'rating': [5, 4, 5, 4, 3, 5, 4]
})

# --- AGENT 1: Content-Based Agent ---
def content_based_recommend(user_movie_title, top_n=3):
    vectorizer = CountVectorizer()
    genre_matrix = vectorizer.fit_transform(movies['genre'])
    cosine_sim = cosine_similarity(genre_matrix)

    index = movies[movies['title'] == user_movie_title].index[0]
    sim_scores = list(enumerate(cosine_sim[index]))
    sim_scores = sorted(sim_scores, key=lambda x: x[1], reverse=True)[1:top_n+1]
    rec_indices = [i[0] for i in sim_scores]
    return list(movies.iloc[rec_indices]['title'])
```

```
# --- AGENT 2: Collaborative Filtering Agent ---
def collaborative_recommend(user_id, top_n=3):
    userRatedMovies = ratings[ratings['user_id'] == user_id]['movie_id']
    similar_users = ratings[ratings['movie_id'].isin(userRatedMovies) & (ratings['user_id'] != user_id)]
    top_movies = (
        similar_users.groupby('movie_id')['rating'].mean()
        .sort_values(ascending=False)
        .head(top_n)
        .index
    )
    return list(movies[movies['movie_id'].isin(top_movies)]['title'])

# --- AGENT 3: Aggregator Agent ---
def multi_agent_recommend(user_id, favorite_movie):
    content_recs = content_based_recommend(favorite_movie)
    collab_recs = collaborative_recommend(user_id)

    # Combine and score
    combined = pd.Series(content_recs + collab_recs).value_counts()
    final_recommendations = combined.head(3).index.tolist()

    return final_recommendations
```

```

import pandas as pd
import numpy as np
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.metrics.pairwise import cosine_similarity
import random

# Sample data
movies = pd.DataFrame({
    'movie_id': [1, 2, 3, 4, 5],
    'title': ['The Matrix', 'John Wick', 'Inception', 'Interstellar', 'The
Dark Knight'],
    'genre': ['Action Sci-Fi', 'Action Thriller', 'Action Sci-Fi', 'Sci-Fi
Drama', 'Action Drama']
})

ratings = pd.DataFrame({
    'user_id': [1, 1, 2, 2, 3, 3, 4],
    'movie_id': [1, 3, 2, 4, 1, 5, 3],
    'rating': [5, 4, 5, 4, 3, 5, 4]
})

# Content-Based Agent
def content_based_recommend(user_movie_title, top_n=1):
    vectorizer = CountVectorizer()
    genre_matrix = vectorizer.fit_transform(movies['genre'])
    cosine_sim = cosine_similarity(genre_matrix)

    index = movies[movies['title'] == user_movie_title].index[0]
    sim_scores = list(enumerate(cosine_sim[index]))
    sim_scores = sorted(sim_scores, key=lambda x: x[1],
reverse=True)[1:top_n+1]
    rec_indices = [i[0] for i in sim_scores]
    return list(movies.iloc[rec_indices]['title'])

```

```

# Collaborative Filtering Agent
def collaborative_recommend(user_id, top_n=1):
    user Rated movies = ratings[ratings['user_id'] ==
user_id]['movie_id']
    similar_users =
ratings[ratings['movie_id'].isin(user Rated movies) &
(ratings['user_id'] != user_id)]
    top_movies = (
        similar_users.groupby('movie_id')['rating'].mean()
        .sort_values(ascending=False)
        .head(top_n)
        .index
    )
    return
list(movies[movies['movie_id'].isin(top_movies)]['title'])

# Simulated user feedback (reward)
def simulate_feedback(user_id, recommended_title):
    # Higher chance of reward if the user rated similar movies
    liked_movies = ratings[ratings['user_id'] ==
user_id]['movie_id']
    liked_titles =
movies[movies['movie_id'].isin(liked_movies)]['title'].tolist()
    return 1 if recommended_title in liked_titles else 0

```



```

# Q-Learning Setup
states = movies['title'].tolist()
actions = ['content', 'collab']
q_table = pd.DataFrame(0, index=states, columns=actions)

alpha = 0.1 # learning rate
gamma = 0.9 # discount factor
epsilon = 0.3 # exploration

# Training the RL agent (Aggregator Agent)
for episode in range(1000):
    user_id = random.choice([1, 2, 3, 4])
    state = random.choice(states)

    # Epsilon-greedy action selection
    if random.uniform(0, 1) < epsilon:
        action = random.choice(actions)
    else:
        action = q_table.loc[state].idxmax()

    # Execute action
    if action == 'content':
        rec = content_based_recommend(state)[0]
    else:
        rec = collaborative_recommend(user_id)[0] if collaborative_recommend(user_id) else state

    # Get reward
    reward = simulate_feedback(user_id, rec)

    # Update Q-table
    old_value = q_table.loc[state, action]
    next_max = q_table.loc[rec].max() if rec in q_table.index else 0
    new_value = old_value + alpha * (reward + gamma * next_max - old_value)
    q_table.loc[state, action] = new_value

```

```
# Final recommender using learned Q-values
def rl_aggregator(user_id, favorite_movie):
    best_action =
q_table.loc[favorite_movie].idxmax()
    if best_action == 'content':
        return
content_based_recommend(favorite_movie)
    else:
        return collaborative_recommend(user_id)
```

https://docs.google.com/document/d/15usu1hkrrRLRjcq_3nCTT0ljEcgiC44iSdvdqrCprk/edit?tab=t.0#heading=h.n8cwc6jfut

https://www.google.com/search?sca_esv=647936d85f11601f&q=agentic+ai+in+recommender+systems&udm=2&fbs=ABzOT_CWdhQLP1FcmU5B0fn3xuWpA-dk4wpBWOgsoR7DG5zJBkzPWUS0OtApXR2914vrjk7XZXfnfKsaRZouQANLhmphEhFjnPez7qSPLLQhF9yWQ3ZrzUbmSVDckVjh88BuJ7Clyev2Ea8wZTZF9TaA47VJJOXvxTfqIxC L23m8h319mHmljuYmW9D9h5-BUK_b-GXXLj4&sa=X&ved=2ahUKEwjW05vGmICNAxWci_0HHYUOFW4QtKgLegQIFhAB&biw=1366&bih=641&dpr=1

<https://medium.com/@official.indrajit.kar/design-considerations-of-advanced-agentic-ai-for-real-world-applications-c5ef317ba1e1>